

Loviny Robin

Dossier Technique

Partie individuel

Epreuve E62

Session 2023

Brevet technicien supérieur:
Système Numérique: Option informatique et réseaux



Projet Supervision Salle de musculation

Projet réalisé par :

Lucas Duhamel
Gildas Midaven
Nicolas BOULANT
Robin LOVINY

Projet encadré par:

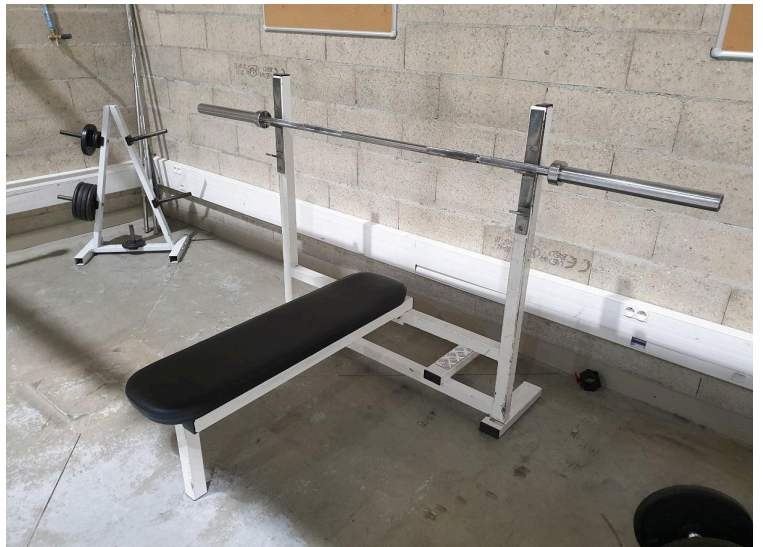
Thomas Drapeau
Sylvain Hardy

Sommaire

Introduction	3
Environnement et cadre du projet	4
Tâches à réaliser	5
Matériel utilisé	6
Diagrammes	7
Configuration du Raspberry Pi	11
Configuration du Wi-Fi	13
Installation/Configuration du Lecteur RFID	14
Programmation Python	16
Conclusion	23
Annexe	24

Introduction

Dans le cadre de notre deuxième année de BTS Systèmes Numériques, option Informatique et Réseaux, nous avons l'opportunité de réaliser un projet. 150 heures nous étaient mises à disposition pour le mener à bien. L'objectif de ce projet est de mettre en pratique nos connaissances et compétences en élaborant un cahier des charges pour la supervision des séances de musculation de la section rugby du lycée Marcel Sembat.



Environnement et cadre du projet

Ce projet est réalisé au sein du lycée Marcel Sembat de Sotteville-lès-Rouen.

Nous avons été encadrés par deux professeurs durant ce projet. Ces personnes sont Mr Hardy et Mr Drapeau.

Objectif :

L'objectif du projet est le développement d'un système de supervision et planification des phases d'entraînement de la section rugby.

Enjeux :

L'enjeu principal du projet est de produire un système qui doit fournir une interface graphique à l'entraîneur lui permettant de planifier une séance d'entraînement, en renseignant les caractéristiques des exercices à effectuer par les différents sportifs.

Il doit également permettre de visualiser facilement l'avancement des exercices tout au long de la séance.

Tâches à réaliser

Les tâches que j'ai eu à réaliser durant ce projet sont:

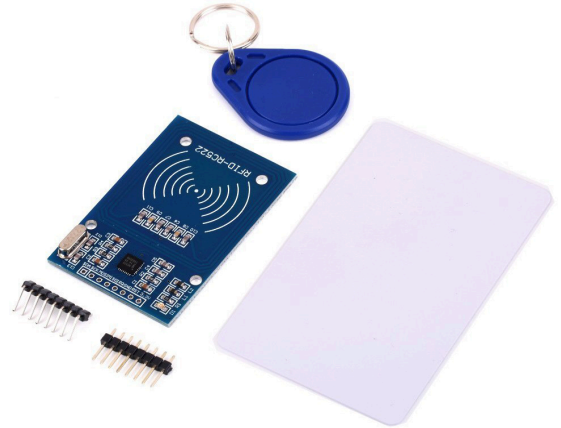
- Développer le service web effectuant la réception et le stockage des données envoyées par les différentes machines
- Concevoir le module IHM utilisé par le sportif
- Mettre en place l'identification du sportif par son badge RFID
- Assurer la transmission des informations via wifi pour stockage dans la base de données du système
- Permettre au sportif d'observer sa progression dans l'exercice.



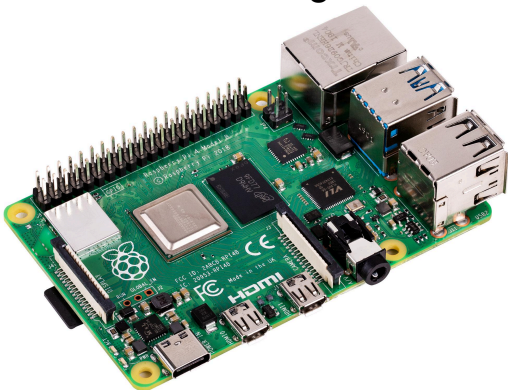
Matériel utilisé



Badge RFID



Lecteur RFID



Raspberry Pi



Afficheur LCD



Programmation en Python



Communication avec la
Base de Données avec MySQL

Diagrammes

Diagramme de blocs interne :

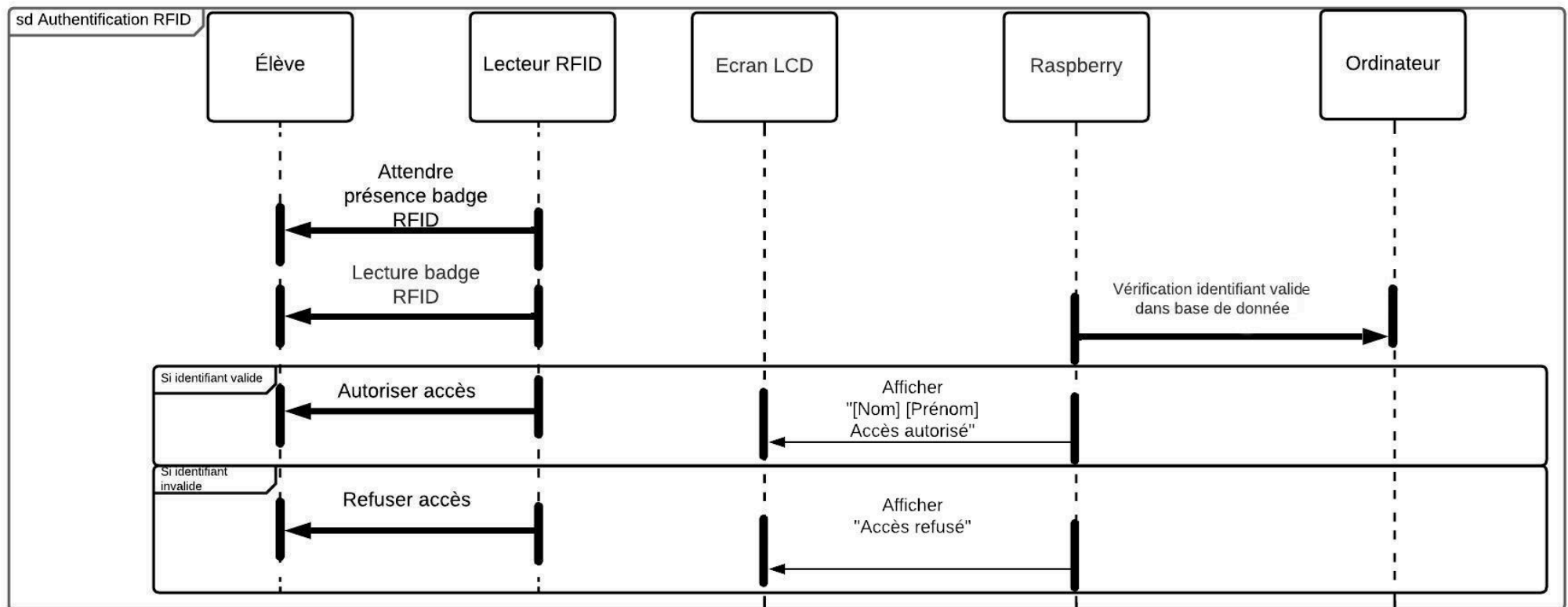


Diagramme de blocs interne :

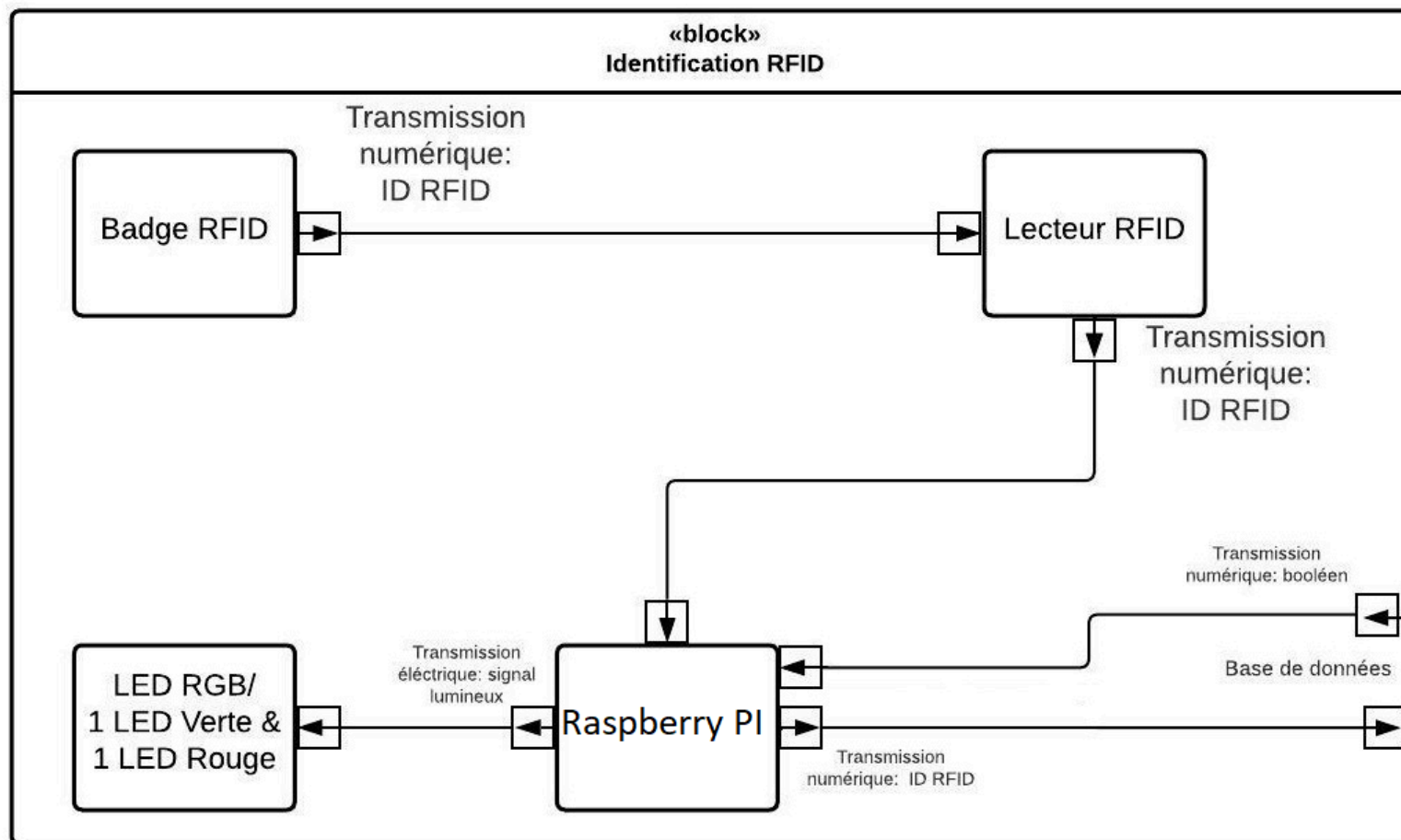


Diagramme de base de donnée :

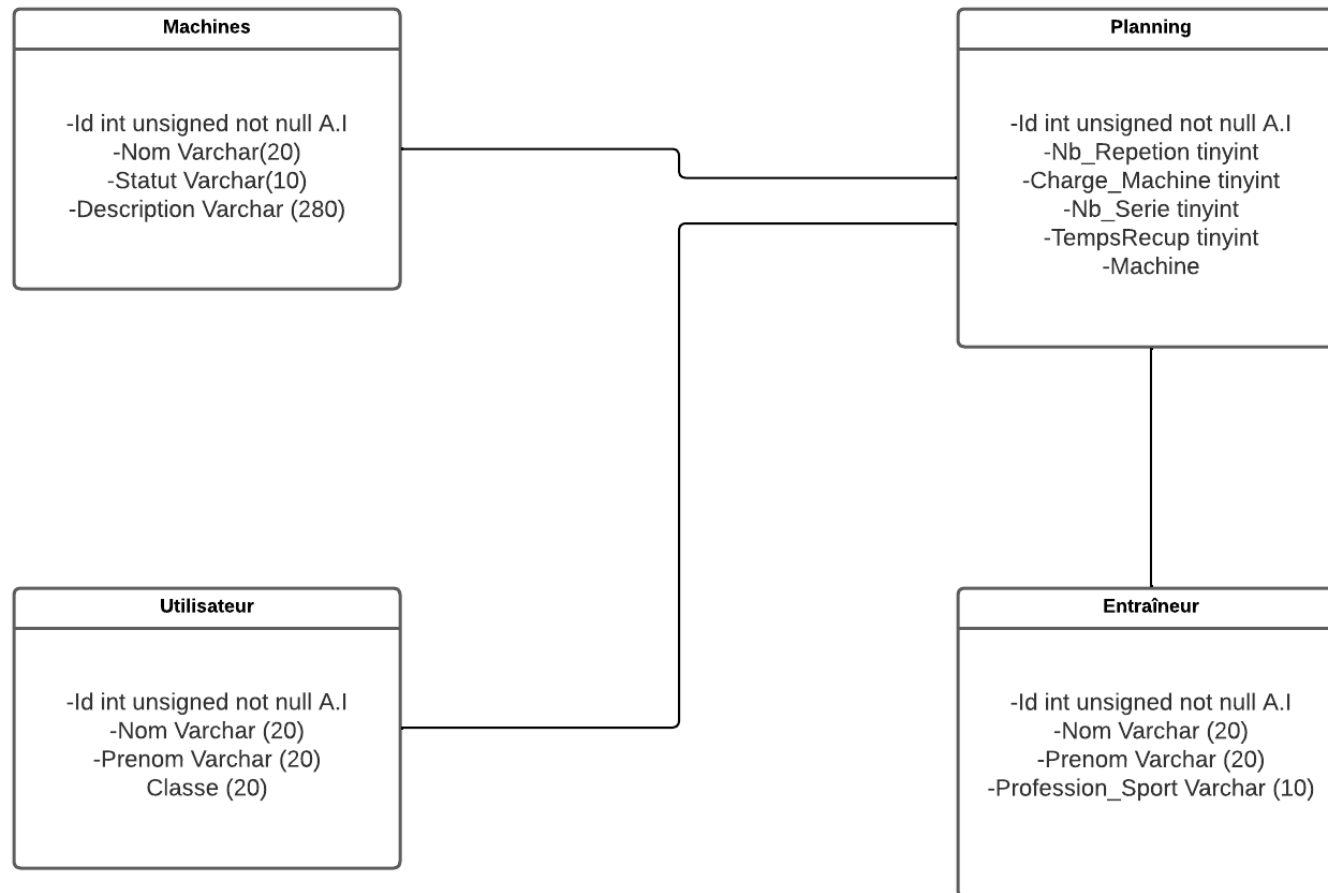
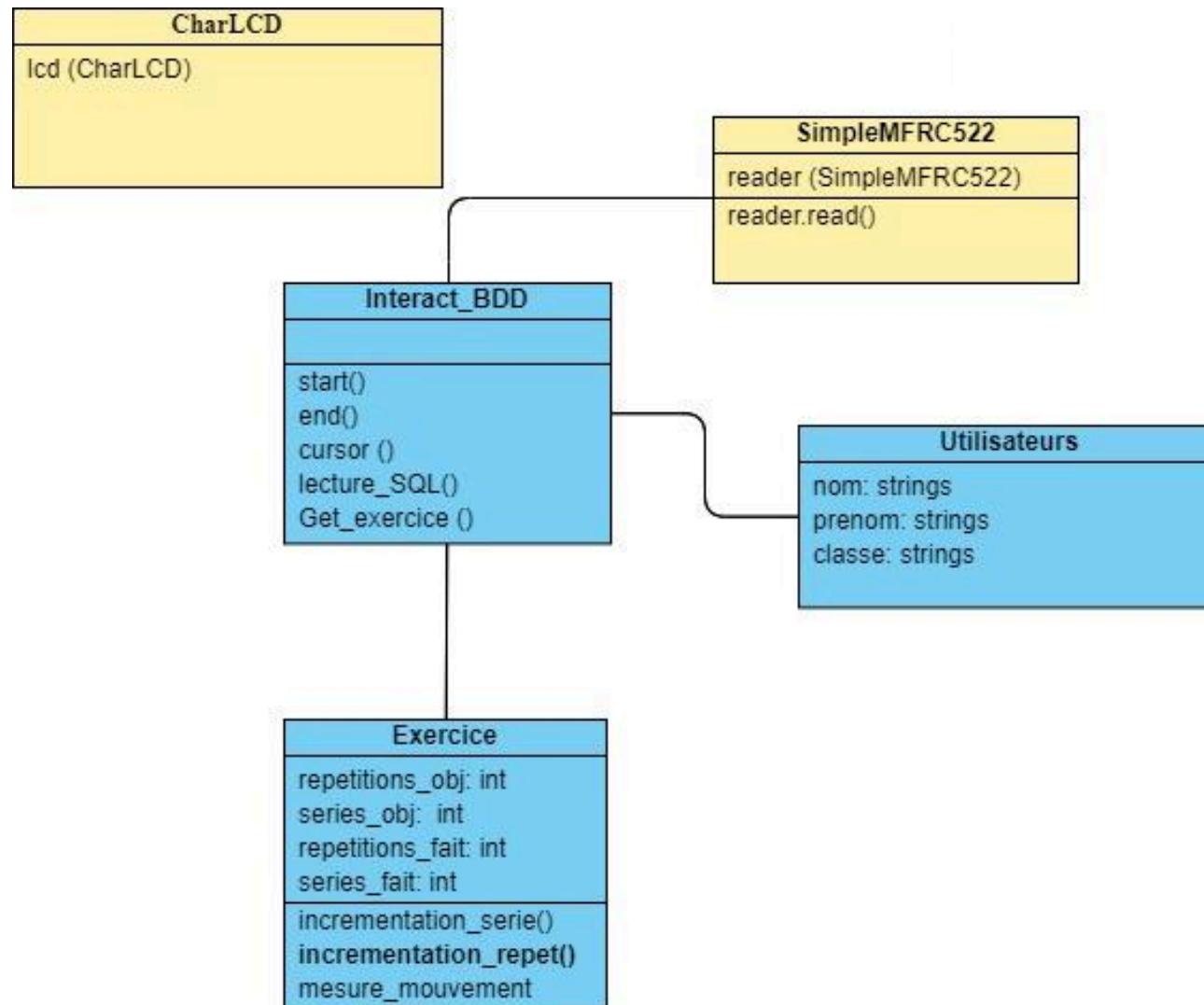
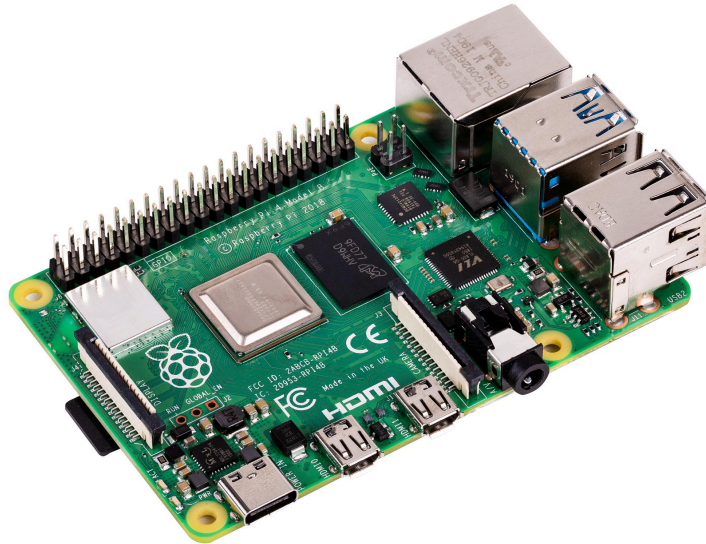


Diagramme de classe :

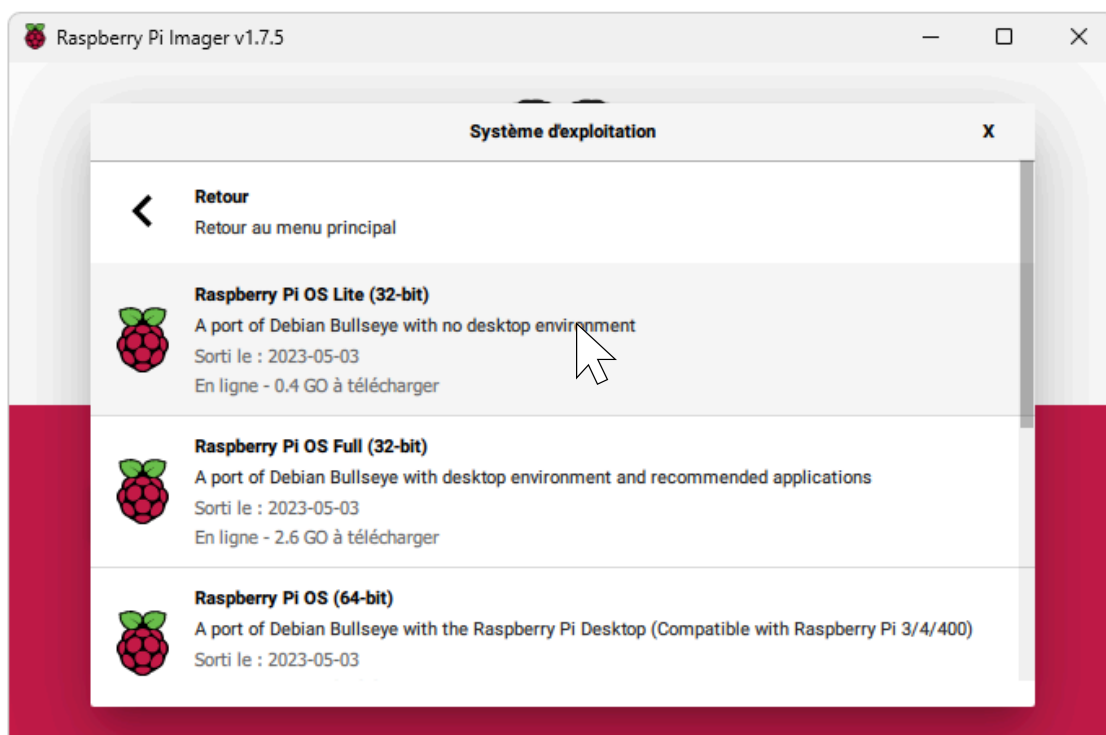


Configuration du Raspberry Pi



Le Raspberry Pi sera utilisé pour piloter le lecteur RFID, réaliser les communications avec la base de données et contrôler l'affichage LCD.

Afin de l'utiliser, il faut d'abord installer un système d'exploitation (SE). Dans le cadre de ce projet, nous utiliserons Raspberry Pi OS, qui est une variante de la distribution Linux Debian conçue spécifiquement pour le Raspberry Pi. Il peut être gravé sur une carte micro SD à l'aide du logiciel officiel Raspberry Pi Imager. Nous n'utilisons que l'interface en ligne de commande ici car le Raspberry Pi ne sera pas relié à un écran, bien qu'une version avec une interface graphique soit également disponible.



Le logiciel nous propose aussi de réaliser des configurations tel que la configuration du Wi-Fi, le SSH ou de l'utilisateur

Règles avancées X

Image customization options pour cette session uniquement

☐ Nom d'hôte raspberrypi Local

☐ Activer SSH

☐ Utiliser un mot de passe pour l'authentification

☐ Authentification via clé publique

Définir `authorized_keys` pour 'btssn2a1':

☒ Définir nom d'utilisateur et mot de passe

Nom d'utilisateur : btssn2a1

Mot de passe : _____

☒ Configurer le Wi-Fi

SSID : RenoL

☐ SSID caché

Mot de passe : ••••••••

☐ Afficher le mot de passe

Pays Wi-Fi : FR

☐ Définir les réglages locaux

Fuseau horaire : Europe/Paris

Type de clavier : US

Règles permanentes

☐ Jouer un son quand terminé

☒ Éjecter le média quand terminé

ENREGISTRER

Une fois l'installation et le premier démarrage du Raspberry Pi terminé, nous allons pouvoir installer différents modules tel que python3 pour la programmation et l'exécution de programme en Python, la librairie python `mysql-connector-python` pour se connecter à une base de donnée et lire des données dans celle-ci depuis un programme Python et la librairie `mfr522` pour contrôler le Lecteur RFID.à partir d'un programme Python.

```
btssn2a1@raspberrypi:~$ sudo pip3 install mysql-connector-python
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting mysql-connector-python
  Downloading mysql_connector_python-8.0.33-cp39-cp39-manylinux1_i686.whl (26.4 MB)
    |-----| 26.4 MB 8.1 MB/s
Collecting protobuf<=3.20.3,>=3.11.0
  Downloading https://www.piwheels.org/simple/protobuf/protobuf-3.20.3-py2.py3-none-any.whl (162 kB)
    |-----| 162 kB 2.4 MB/s
Installing collected packages: protobuf, mysql-connector-python
Successfully installed mysql-connector-python-8.0.33 protobuf-3.20.3
btssn2a1@raspberrypi:~$ sudo pip3 install spidev
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: spidev in /usr/lib/python3/dist-packages (3.5)
btssn2a1@raspberrypi:~$ sudo pip3 install mfr522
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting mfr522
  Downloading https://www.piwheels.org/simple/mfr522/mfr522-0.0.7-py3-none-any.whl (18 kB)
Collecting RPi.GPIO
  Downloading RPi.GPIO-0.7.1.tar.gz (29 kB)
Requirement already satisfied: spidev in /usr/lib/python3/dist-packages (from mfr522) (3.5)
Building wheels for collected packages: RPi.GPIO
  Building wheel for RPi.GPIO (setup.py) ... done
  Created wheel for RPi.GPIO: filename=RPi.GPIO-0.7.1-cp39-cp39-linux_i686.whl size=66357 sha256=6063c89c10cfb6fe86bed54be7812e33096a366a7af72dcd51fb13f0d377e9b
  Stored in directory: /root/.cache/pip/wheels/b4/65/39/789a186465c4823ffa930b950af77c86f1494adf1c8e21ac1a
Successfully built RPi.GPIO
Installing collected packages: RPi.GPIO, mfr522
Successfully installed RPi.GPIO-0.7.1 mfr522-0.0.7
btssn2a1@raspberrypi:~$
```

Configuration du Wi-Fi

Afin d'assurer une connexion sans fil du Raspberry Pi, nous utiliserons la communication Wi-Fi pour le Raspberry Pi.

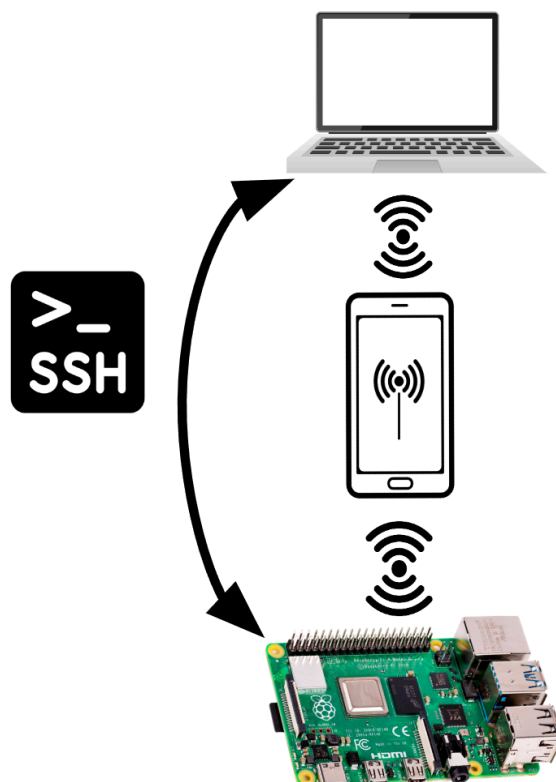
Le Wi-Fi est inclus sur les modèles 3 et 4 des Raspberry Pi. Le modèle utilisé ici est un Raspberry Pi 4, ce qui signifie qu'il dispose également de la connectivité Wi-Fi intégrée.

Il est possible de configurer le Wi-Fi dans le fichier `/etc/wpa_supplicant/wpa_supplicant.conf/`

```
GNU nano 5.4 /etc/wpa_supplicant/wpa_supplicant.conf *
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=FR

network = {
    ssid="ssid" #ssid du réseau
    psk="motDePasse" #mot de passe du réseau
}
```

J'ai ensuite réalisé une liaison entre le Raspberry Pi et un PC portable à partir d'un point d'accès d'un téléphone portable afin de vérifier le fonctionnement du Wi-Fi. J'ai réussi à établir une connexion SSH depuis l'ordinateur portable, ce qui me permet de contrôler le Raspberry Pi sans utiliser de connexion filaire et de bien vérifier que le Wi-Fi fonctionne sur le Raspberry Pi.

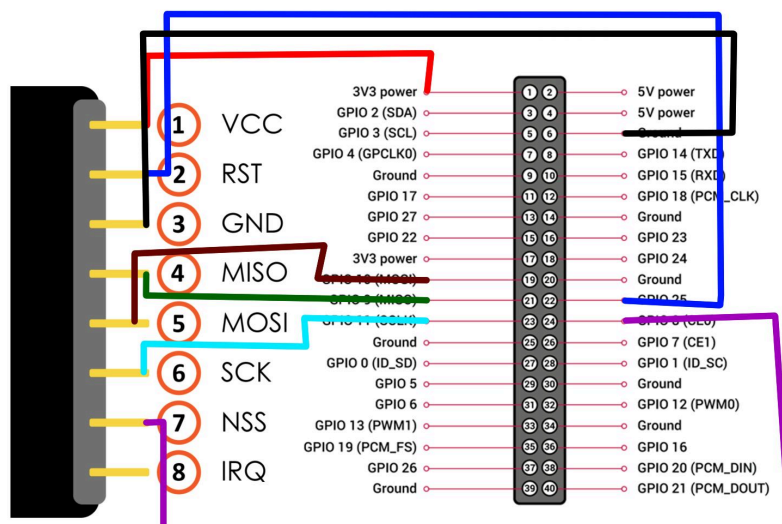


Installation/Configuration du Lecteur RFID

L'utilisation du lecteur RFID avec le Raspberry Pi nécessite de le connecter aux broches du port GPIO du Raspberry Pi.

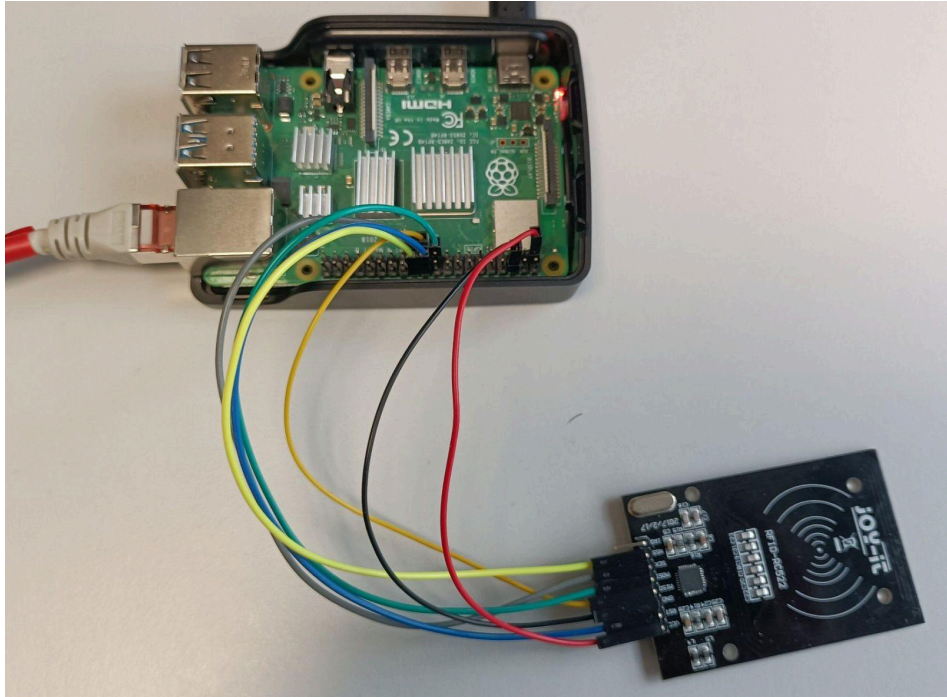
Le lecteur RFID doit être relié au Raspberry Pi comme décrit ci-dessous:

RFID	Raspberry Pi
VCC	1 (3,3 V)
RST	22 (GPIO 25)
GND	6 (GND)
MISO	21 (GPIO 9)
MOSI	19 (GPIO 10)
SCK	23 (GPIO 11)
NSS	24 (GPIO 8)

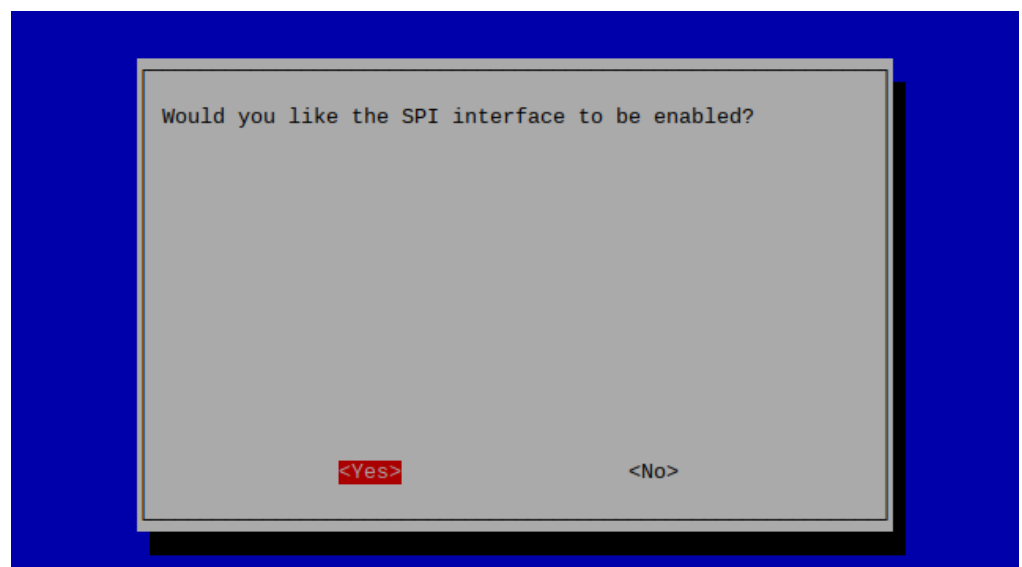
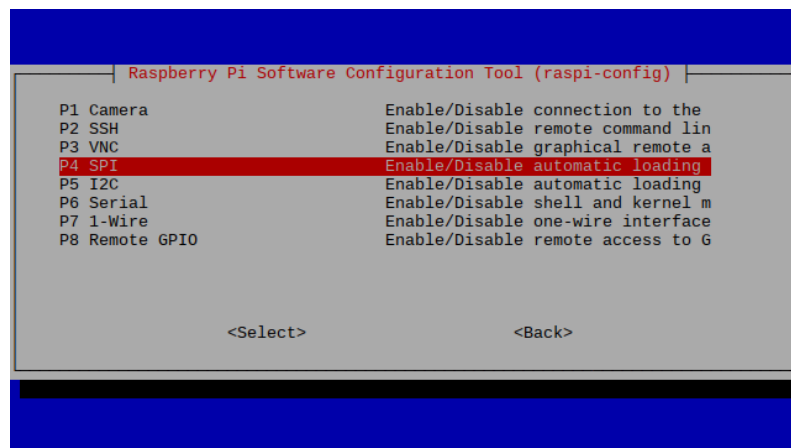
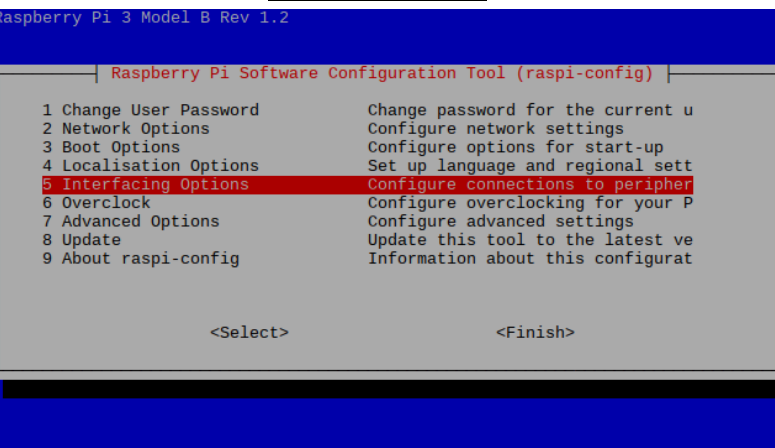


VCC	Alimentation
RST	Broche de réinitialisation (Reset)
GND	Ground
MISO	Master Input Slave Output: Réception d'information du Lecteur
MOSI	Master Output Slave Input: Envoie d'information au Lecteur
SCK	Signal d'horloge
NSS	Information non trouvé

1 (3,3 V)	Alimentation 3.3 V
22 (GPIO 25)	General Purpose Input Output: Broche d'envoi et de réception d'information
6 (GND)	Ground
21 (GPIO 9)	General Purpose Input Output
19 (GPIO 10)	General Purpose Input Output
23 (GPIO 11)	General Purpose Input Output
24 (GPIO 8)	General Purpose Input Output



Il faut aussi activer l'interface SPI du Raspberry Pi utilisé par le lecteur avec la commande **sudo raspi-config**.



Programmation Python

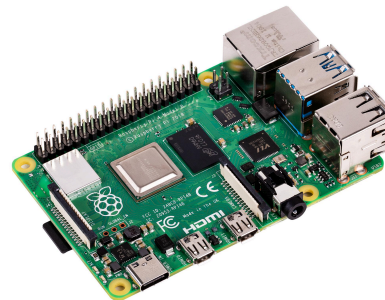


Cette partie traitera uniquement de la programmation du Lecteur RFID, car je n'ai pas encore eu le temps de réaliser le programme pour l'afficheur LCD.

Les programmes réalisés utilisent principalement la librairie du Lecteur RFID `mfr522` et la librairie de communication avec la base de données MySQL `mysql-connector-python`.



Certains programmes ne sont pas définitifs, notamment la fonction `counter`, la fonction `get_machine`, ainsi que le programme principal (`main`).



Classe et fonctions d'interactions avec la base de données: déclarations de librairies et démarrage de la connection avec la base de données:

```
#!/usr/bin/env python
#-- coding: utf-8 --
import mysql.connector
import sys
import time
from time import sleep

class interact_BDD:
    def __init__(self):
        self.cnx = None

    def start(self):
        try:
            # connection avec la base de données
            print("Connection a la base de donnees...")
            self.cnx = mysql.connector.connect(user='Projet', password='btssnir', #connection avec
            identifiants et
                                                    #mot de passe
                                                    host='172.18.215.123', #adresse IP de la base de
données
                                                    database='projetmuscu') #nom

            print("Connecte avec succes")
            return self.cnx
        except mysql.connector.Error as err: #affichage de l'erreur si connexion non établie
            print("Echec de la connection: {}".format(err))
            return None
```

Arrêt de la connexion avec la base de données:

```
def end(self):  
    # arrêt de la connexion  
    self.cnx.close()  
    print("Arret de la connection")
```

Recherche de l'identifiant (ID) dans la base de données:

```
def LectureSQL(self, id):  
    #Lecture des données dans la table en fonction de l'ID  
    cursor = self.cnx.cursor()  
    query = ("SELECT Nom, Prenom, id FROM utilisateur WHERE id_carte = %s") #Création de la requête SQL.  
    "%s" = id de la carte  
    cursor.execute(query, (id,)) #Exécution de la requête avec id comme valeur  
    result = cursor.fetchone() #résultat enregistré dans result  
    if result is None:          #Message d'erreur si donnée introuvable  
        print("Donnees introuvables")  
        sys.exit()  
    else:  
        print("Nom: {}".format(result[0]))    #Ecriture des resultat obtenu  
        print("Prenom: {}".format(result[1]))  
        #print ("ID: {}".format(result[2]))  
        return result[2]    #envoi de l'ID utilisateur au main
```

Recherche des informations sur l'exercice en fonction de l'utilisateur et de la machine dans la base de données:

```
def get_machine(self, id_user): # Lecture des information de l'exercice en fonction de la machine et de
l'utilisateur
    cursor = self.cnx.cursor()
    request = ("SELECT Nom_Machine, NbRepetition, NbSeries, TempsRecuperation from planning INNER JOIN
machines ON planning.MachineId = machines.id WHERE planning.UtilisateurId = %s") #Création de la requête SQL.
"%s" = id de l'utilisateur
    cursor.execute(request, (id_user,)) #Exécution de la requête avec id_user comme valeur
    result = cursor.fetchone() #résultat enregistré dans result
    if result is not None:
        print ("Type exercice: {}".format(result[0])) #Ecriture des résultat obtenu
        print ("Nombre de series: {}".format(result[2]))
        print ("Nombre de repetition: {}".format(result[1]))
        print ("Temps de recuperation: {}".format(result[3]))
        Nmbrerepubet = result [1] #Enregistrement des résultat obtenu dans différentes variables
        Nmbreseries = result [2]
        TempsRecup = result [3]
        self.counter(Nmbrerepubet, Nmbreseries, id_user) #Envoi des résultat obtenu à la fonction counter
    else:
        #Message d'erreur si aucune informations trouvées
        print("Aucune machine atittre trouve.")
        sys.exit()
```

Programme d'affichage de compteur de Séries:

```
def counter (self, Nnbrerepet, Nmbreseries, id_user): #Execution avec les données de get_machine
    cursor = self.cnx.cursor()
    request = ("SELECT NbRepetition, NbSeries, SerieEffectuee from planning WHERE UtilisateurId = %s")
#initialisation de la
requête SQL
    while (Serieeffect != Nmbreseries): #Tant que le nombre de Série réalisé n'est pas égal aux Nombre de
séries à réaliser
        cursor.execute(request, (id_user,)) #Exécution de la requête SQL
        result = cursor.fetchone() #Résultat enregistré dans result
        Nmbreseries = result[1]      #Nombre de séries enregistrés dans NmbreSeries
        Nnbrerepet = result[0]       #Nombre de répétition enregistrés dans Nnbrerepet
        Serieeffect = result[3]      #Nombre de séries effectuées dans SeriesEffect
        print ("Series effectue: {}".format(result[2])) #Affichage du nombre de séries effectuée
        print ("Series a realiser: {}".format(result[1])) #Affichage du nombre de séries à réaliser
        time.sleep(0.5) #pause de 0.5 sec avant reprise au début de la boucle
```

Programme principal (main):

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import mysql.connector #importation des librairies et de la classe cmd_BDD
import time
from mfrc522 import SimpleMFRC522
from time import sleep
import cmd_BDD

BDD = cmd_BDD.interact_BDD() #instanciation de la classe interact_BDD
reader = SimpleMFRC522() #instanciation de la librairies du Lecteur RFID
BDD.start() #connection à la base de donnée
try:
    while True:
        print("Placez le badge sur le lecteur:")
        id, nom = reader.read() #Lecture du badge RFID et enregistrement des données
        #time.sleep(2)
        id_user = BDD.LectureSQL(id) #Envoi de l'ID carte à la fonction LectureSQL et récupération du
résultat obtenu
        time.sleep(2)
        BDD.get_machine(id_user) #Envoi de l'ID utilisateur à la fonction get_machine
        time.sleep(2)
except KeyboardInterrupt: #Si arrêt demandé avec Ctrl+C
    print("\nArret du programme demande...")
    BDD.end() #Arrêt de la connexion
```

Exemple de sortie du programme:

```
Placer le badge sur le lecteur:  
Nom: Loviny  
Prenom: Robin  
Type d'exercice: Banc d'appui large  
Nombre de séries: 3  
Nombre de repetition: 10  
Temps de recuperation: 30  
#Boucle tant que les séries ne sont pas terminés  
Series effectuee: Series Effectuée  
Series a realiser: 3
```

Conclusion

Pour conclure mon expérience lors de la réalisation de ce projet, j'ai grandement apprécié travailler sur ma partie, car cela m'a permis d'approfondir mes connaissances sur l'utilisation du Raspberry Pi, notamment en ce qui concerne le Wi-Fi, l'utilisation du port GPIO, ainsi que sur Linux

J'ai aussi aimé faire la programmation de ce projet en Python car c'est un langage dans lequel je n'avais pas une grande expérience auparavant. Je suis alors parvenu à approfondir mes connaissances. Je me suis davantage intéressé à ce langage de programmation et j'ai aussi pu réparer certaines de mes lacunes.

J'ai également appliqué mes connaissances en SQL dans lequel j'avais déjà des bases solides. J'ai pu corriger certaines erreurs et j'ai découvert d'autres utilisations de ce langage que je ne connaissais pas.

J'ai aussi apprécié travailler en groupe car malgré ma timidité je suis parvenu à prendre davantage confiance en moi dans un projet en équipe, et j'ai aimé mettre en commun mon travail avec mes camarades.

Annexe

Manuel du Lecteur RFID utilisé:

<https://urlz.fr/maM5>

Configuration et installation du Lecteur RFID par PiMyLifeUp (Anglais):

<https://urlz.fr/maM8>

Comprendre les ports GPIO par Paul Mcwhorter (Vidéo en Anglais):

<https://urlz.fr/maNH>

Documentation officiel de Raspberry Pi:

<https://urlz.fr/maNK>

Site de Raspberry Pi OS:

<https://urlz.fr/lGq9>

Introduction	3
Environnement et cadre du projet	4
Tâches à réaliser	5
Matériel utilisé	6
Diagrammes	7
Configuration du Raspberry Pi	11
Configuration du Wi-Fi	13
Installation/Configuration du Lecteur RFID	14
Programmation Python	16
Conclusion	23
Annexe	24

